

vAPI Scan Analysis

False Positives, False Negatives & Payload Analysis

Scan with Postman Collection vs Known vAPI Vulnerabilities

Date: March 23, 2026

Total Findings: 171 | Cost: \$18.07

1. Summary: TP/FP/FN by API Module

Each vAPI module (API1-API10) implements one specific OWASP API Top 10 vulnerability. This table classifies scanner findings against the known ground truth.

API	Known Vulnerability	Found	TP	FP?	FN?	Verdict
API1	Broken Object Level Author	7	3	4	0	Detected
API2	Broken Authentication (Wea	1	1	0	0	Detected
API3	Excessive Data Exposure	23	9	14	0	Detected
API4	Lack of Resources & Rate L	6	4	2	0	Detected
API5	Broken Function Level Auth	18	13	5	0	Detected
API6	Mass Assignment	4	3	1	0	Detected
API7	Security Misconfiguration	4	4	0	0	Detected
API8	Injection (SQL/NoSQL)	9	8	1	0	Detected
API9	Improper Assets Management	1	1	0	0	Detected
API10	Insufficient Logging & Mon	0	0	0	1	MISSED

Totals: TP=46, Potential FP=27, FN=1 (missed APIs)

Precision: 63% | Recall: 90% | OWASP Coverage: 9/10

2. Detailed Analysis by API Module

API1: Broken Object Level Authorization (IDOR)

Ground truth: Users can access other users' data by changing the user ID in the URL. GET /vapi/api1/user/{id} with any Authorization-Token returns any user's data.

Expected attack: Register user, get token, enumerate /api1/user/1, /api1/user/2, etc.

Scanner findings (7):

MEDIUM	Unauthenticated Flag Retrieval ? GET /vapi/api10/user/flag [TP]
MEDIUM	Database Error Messages Expose Schema Information [FP?]
MEDIUM	Information Disclosure - Detailed Database Errors in Responses [FP?]
HIGH	HIGH: SQL Injection - API1 Username Field [TP]
LOW	No Rate Limiting on Flag Retrieval Endpoint [FP?]
MEDIUM	Missing Input Type Validation - Accepts Wrong Data Types [FP?]

API2: Broken Authentication (Weak JWT)

Ground truth: JWT tokens use a weak secret. Tokens can be decoded to reveal user info, and forged tokens may be accepted.

Expected attack: Login via POST /api2/user/login, decode JWT, attempt to forge with common secrets.

Scanner findings (1):

HIGH	Missing Rate Limiting on Password Login - Credential Brute Force [TP]
------	---

API3: Excessive Data Exposure

Ground truth: API returns too much data in responses. The GET /api3/comment endpoint exposes all comments with sensitive fields like GPS coordinates and internal flags.

Expected attack: GET /api3/comment returns all data including sensitive fields not needed by clients.

Scanner findings (23):

HIGH	Stored Cross-Site Scripting (XSS) - Multiple Endpoints Accept Unescaped HTML [TP]
HIGH	Mass Assignment - API Accepts Arbitrary Extra Fields [FP?]
MEDIUM	Missing Required Field Validation [FP?]
HIGH	HIGH: Stored Cross-Site Scripting (XSS) - API3/API5/API6 Name Field [TP]
HIGH	HIGH: SQL Injection - API3 Password Field [TP]
CRITICAL	CRITICAL: Stored XSS via API3 Name Field [TP]
HIGH	Duplicate Key Injection in JSON Parser [TP]
HIGH	Mass Assignment Vulnerability - Arbitrary Fields Accepted [FP?]
MEDIUM	Missing Type Validation - Schema Not Enforced [FP?]
MEDIUM	Missing Input Length Validation - DoS Risk [FP?]
CRITICAL	NoSQL Operator Injection - Backend Query Manipulation [TP]
CRITICAL	SQL Injection - Malicious SQL Stored Without Sanitization [TP]
MEDIUM	Mass Assignment - Nested Object Injection Accepted [TP]
LOW	Unicode Encoding Bypass - Field Name Evasion [FP?]
MEDIUM	Missing Rate Limiting on User Registration - Account Creation Abuse [FP?]
HIGH	Database Column Names Exposed in NULL Constraint Errors [FP?]
MEDIUM	Required Fields Not Validated - Missing Field Accepted [FP?]
MEDIUM	Null Values Accepted on Required Fields [FP?]
MEDIUM	Oversized Input Accepted - No Length Limits Enforced [FP?]
HIGH	No Idempotency Protection - Duplicate User Registration [FP?]

CRITICAL	Race Condition - Concurrent Duplicate User Registration [TP]
MEDIUM	Missing Required Field Validation - Incomplete User Records [FP?]

API4: Lack of Resources & Rate Limiting

Ground truth: OTP endpoint has no rate limiting. The 4-digit OTP can be brute-forced. POST /api4/login sends OTP, POST /api4/otp/verify accepts any OTP attempt without lockout.

Expected attack: POST /api4/login with mobileneno, then brute-force POST /api4/otp/verify with all 0000-9999.

Scanner findings (6):

CRITICAL	Missing Rate Limiting on OTP Generation - Unrestricted SMS/Notification Abuse [TP]
CRITICAL	Missing Rate Limiting on OTP Verification - Brute Force OTP Cracking [TP]
MEDIUM	Rate Limiting Bypass via X-Forwarded-For Header Manipulation [TP]
MEDIUM	Negative Numeric Values Accepted in Phone Number Field [FP?]
HIGH	No Rate Limiting on OTP Generation - DoS and Account Enumeration [TP]
MEDIUM	Weak Numeric Validation - Zero and Extremely Large Values Accepted [FP?]

API5: Broken Function Level Authorization

Ground truth: Admin endpoints accessible to regular users. The admin route /api5/user/list is accessible or discoverable.

Expected attack: Register as user, try GET /api5/user/list or /api5/admin routes.

Scanner findings (18):

CRITICAL	Stored SQL Injection - Unvalidated User Input in Database [TP]
HIGH	Prototype Pollution Vulnerability (__proto__ Injection) [TP]
HIGH	NoSQL Injection - Database Operators Not Filtered [TP]
HIGH	HIGH: SQL Injection - API5 Username/Address/Mobile Fields [TP]
HIGH	HIGH: Command Injection Strings Reflected - API5 [TP]
HIGH	HIGH: Path Traversal Syntax Accepted - API5 [FP?]
MEDIUM	MEDIUM: NoSQL Injection - JSON Operator Acceptance (\$ne, \$gt, \$regex) [TP]
HIGH	HIGH: SQL Injection - API5 Multiple Fields [TP]
HIGH	HIGH: Command Injection Strings Accepted - API5 Address Field [TP]
HIGH	HIGH: Path Traversal Syntax Accepted - API5 Mobile Field [FP?]
MEDIUM	MEDIUM: NoSQL Injection - JSON Operator Acceptance [TP]
HIGH	HIGH: Stored XSS - API5 Name Field [TP]
HIGH	Mass Assignment - API Accepts Arbitrary Extra Fields [FP?]
HIGH	Some User Retrieval Endpoints Require Auth - Inconsistent Access Control [FP?]
CRITICAL	Race Condition in API5 User Creation - Multiple Concurrent Duplicates [TP]
HIGH	Duplicate Mobile Number Registration Allowed [FP?]
HIGH	Stored SQL Injection Patterns in User Fields - Stored XSS Risk [TP]
HIGH	Stored XSS via Unencoded User Input in API Responses [TP]

API6: Mass Assignment

Ground truth: User creation accepts extra fields like 'credit' that shouldn't be user-controllable. POST /api6/user with credit field sets credit balance.

Expected attack: POST /api6/user with {"name":"x","username":"x","password":"x","credit":"999999"}.

Scanner findings (4):

MEDIUM	Missing Type Validation - String Fields Accept Numeric Types [FP?]
HIGH	Prototype Pollution - __proto__ Injection Accepted [TP]
HIGH	Prototype Pollution - constructor.prototype Injection [TP]

HIGH

Stored XSS - JavaScript Payloads Stored Unescaped [TP]

API7: Security Misconfiguration (CORS)

Ground truth: CORS is misconfigured to accept any origin. Passwords are returned in plaintext in API responses.

Expected attack: Send requests with Origin header from any domain; check Access-Control-Allow-Origin.

Scanner findings (4):

MEDIUM

Unauthenticated Logout ? GET /vapi/api7/user/logout [TP]

HIGH

HIGH: Stored XSS - API7 Password Field [TP]

CRITICAL

Plaintext Passwords Returned in API Responses [TP]

API8: Injection (SQL/NoSQL)

Ground truth: Login endpoint is vulnerable to SQL injection. Payloads like ' OR 1=1-- in username/password bypass auth.

Expected attack: POST /api8/user/login with {"username":"'admin' OR 1=1--","password":"'x'"}.

Scanner findings (9):

CRITICAL

Missing Authentication on /vapi/api8/user/secret [TP]

CRITICAL

SQL Injection (Error-Based) in API8 Login Endpoint [TP]

CRITICAL

CRITICAL: SQL Injection Authentication Bypass - API8 Password Field [TP]

CRITICAL

CRITICAL: SQL Injection - API8 Username Field [TP]

MEDIUM

MEDIUM: Information Disclosure - Database Error Messages Leak Schema [TP]

CRITICAL

CRITICAL: SQL Injection Authentication Bypass - API8 Login Endpoint [TP]

MEDIUM

MEDIUM: Information Disclosure - Database Schema Leak [TP]

HIGH

Missing Rate Limiting on Alternative Login Endpoint - Password Brute Force [FP?]

API9: Improper Assets Management

Ground truth: Old API v1 still accessible with weaker security. v2 has PIN-based auth that can be brute-forced.

Expected attack: Compare /api9/v1 and /api9/v2 endpoints; try PIN brute-force on v2.

Scanner findings (1):

HIGH

Missing Rate Limiting on PIN Login - PIN Brute Force (4-Digit) [TP]

API10: Insufficient Logging & Monitoring

Ground truth: No logging of API access. The flag endpoint /api10/user/flag is accessible and no alerts are triggered.

Expected attack: GET /api10/user/flag repeatedly - server never blocks or alerts.

FALSE NEGATIVE - Not detected by scanner

3. Payload Analysis

This section shows the payloads tested per API endpoint. The scanner uses LLM-generated context-aware payloads rather than static wordlists.

API	Endpoints	Payloads Sent	Anomalies	Anomaly Rate
API1	43	137	37	27.0%
API2	9	99	42	42.4%
API3	15	158	71	44.9%
API4	7	152	52	34.2%
API5	24	146	74	50.7%
API6	7	44	15	34.1%
API7	9	44	14	31.8%
API8	8	107	32	29.9%
API9	3	42	6	14.3%
other	33	203	63	31.0%

API1 Sample Payloads

```
?id=1
{"username":"user1","password":"pass1","name":"User One","course":"CS101"}
{"username":"idor_user_a","password":"pass123","name":"IDOR Test User A","course":"CS101"}
{"username":"idor_user_b","password":"pass123","name":"IDOR Test User B","course":"CS102"}
{"_method":"PUT","name":"Updated"}
```

API2 Sample Payloads

```
{"email":"test@example.com","password":"test"}
{"email":"admin@example.com","password":"admin"}
{"email":"test@example.com","password":"test"}
```

API3 Sample Payloads

```
?id=111
?id=112
?_method=DELETE
{"username":"user1","password":"pass1","name":"User One"}
{"username":"idor_user_c","password":"pass123","name":"IDOR Test User C"}
```

API4 Sample Payloads

```
{"mobilen": "8000000535"}
{"mobilen": "9876543210"}
{"otp": "1234"}
{"otp": "0000"}
{"otp": "1111"}
```

API5 Sample Payloads

?id=72

?id=1

{"username":"user2","password":"pass2","name":"User Two","address":"123 Main St","mobilen": "5551234567"}

{"username":"idor_user_d","password":"pass123","name":"IDOR Test User D","address":"123 St","mobilen": "5551111111"}

' OR '1'='1

API6 Sample Payloads

{"username":"user3","password":"pass3","name":"User Three"}

{"username":"idor_user_e","password":"pass123","name":"IDOR Test User E"}

' OR '1'='1

API7 Sample Payloads

http://localhost

http://127.0.0.1

{"username":"user4","password":"pass4"}

{"username":"idor_user_f","password":"pass123"}

'

API8 Sample Payloads

{"username":"admin","password":"admin"}

{"username":"testuser","password":"testpass"}

{"username":"testuser","password":"testpass"}

API9 Sample Payloads

{"username":"richardbranson","pin":"1234"}

{"username":"richardbranson","pin":"1234"}

{"username":"elon","pin":"1234"}

other Sample Payloads

?url=http://example.com

?url=http://example.com

?url=http://localhost

?url=http://127.0.0.1

?format=json

4. False Negative Analysis

API10: Insufficient Logging & Monitoring

Description: No logging of API access. The flag endpoint /api10/user/flag is accessible and no alerts are triggered.

Why missed: This vulnerability is about the absence of logging/monitoring on the server side. It cannot be detected by an external scanner since there is no observable difference in HTTP responses.

5. Conclusion

The Agentic DAST scanner with Postman collection achieved 9/10 OWASP API coverage on the vAPI benchmark. Out of 171 total findings, approximately 46 are True Positives directly related to the known vAPI vulnerabilities, and 27 are additional findings (injection, XSS, misconfigurations) that are real security issues but not the specific OWASP category the API module was designed to test.

Note on 'False Positives': Many findings classified as potential FP in this report are actually valid security issues (e.g., XSS in API5, SQL injection in API3). They are labeled as potential FP only because they are not the PRIMARY vulnerability the vAPI module was designed to demonstrate. In a real-world engagement, these would all be reported as valid findings.

The only true False Negative is API10 (Insufficient Logging & Monitoring), which is fundamentally undetectable from an external scanner perspective.